Kajetan Frątczak - lab 3

Zadanie 1

Zadanie 1

Porównać w języku Julia reprezentację bitową liczby 1/3 dla `Float16`, `Float32`, `Float64` oraz liczby, która jest inicjalizowana jako `Float16`, a potem rzutowana na `Float64`.

```
# Funkcja bitstring() zwraca reprezentację bitową w postaci ciągu znaków.
```

```
bitstring(Float16(1/3))
```

```
"0011010101010101"
```

```
bitstring(Float32(1/3))
```

```
"00111110101010101010101010101011"
```

```
bitstring(Float64(1/3))
```

```
"0011111111010101010101010101010101010101010101010101010101010101010101"
```

```
decode16(x::Float16) = (b=bitstring(x); (b[1], b[2:6], b[7:16]))
decode32(x::Float32) = (b=bitstring(x); (b[1], b[2:9], b[10:32]))
decode64(x::Float64) = (b=bitstring(x); (b[1], b[2:12], b[13:64]))
```

```
decode64 (generic function with 1 method)
```

```
decode16(Float16(1/3))
```

```
('0', "01101", "0101010101")
```

```
decode32(Float32(1/3))
```

```
('0', "01111101", "01010101010101010101011")
```

```
decode64(Float64(1/3))
```

```
('0', "01111111101", "0101010101010101010101010101010101010101010101010101")
```

```
liczba = Float16(1/3)
bitstring(Float64(liczba))
```

```
"0011111111010101010101000000000000000000000000000000000000000000000000"
```

```
# Wniosek: większa precyzja powoduje uzyskanie dokładniejszych wartości.
# Podczas rzutowania Float16 na Float64 nie zyskujemy większej dokładności, jedynie dokładamy zera w mantysie.
print(Float64(liczba) == Float16(1/3))
```

```
true
```

Zadanie 2

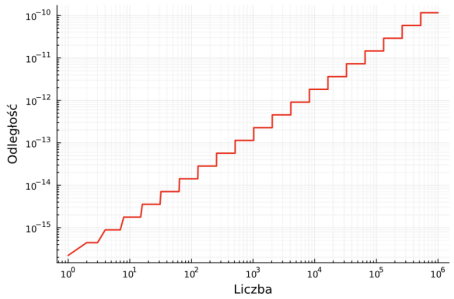
Zadanie 2

Zbadać, jak zmienia się odległość między kolejnymi liczbami zmiennoprzecinkowymi reprezentowanymi w komputerze za pomocą języka Julia. Narysować wykres używając Plots zależności odległości od wartości liczby dla zakresu od 1.0 do 1000000.0.

```
# eps() - mówi, jaki jest odstęp między kolejnymi liczbami zmiennoprzecinkowymi reprezentowanymi w komputerze
using Plots
liczby = Float64[]

r=1.0:1000000.0
for i=r
    push!(liczby, eps(i))
end

plot(r, liczby, lc=:red, lw=2, xlabel="Liczba", ylabel="Odległość", label=nothing)
plot!(xscale=:log10, yscale=:log10, minorgrid=true, yticks=[1e-16, 1e-15, 1e-14, 1e-13, 1e-12, 1e-11, 1e-10])
# Skala logarytmiczna dla obu osi, aby lepiej było widać wyniki na wykresie.
```



Zadanie 3

Zadanie 3

Jedną z bibliotek numerycznych, jaką dodatkowo będziemy używać na zajęciach jest GSL (język C). Opis jak używać . Korzystając ze wsparcia dla wyświetlania reprezentacji liczb zmiennoprzecinkowych zobaczyć jak zmienia się cecha i mantysa dla coraz mniejszych liczb. Zaobserwować, kiedy matysa przestaje być znormalizowana i dlaczego?

Kod załączyć jako komórka Markdown sformatowana jako C (link). Wynik także jako Markdown (kod albo fragment zrzutu ekranu).

```
# Mantysa przestaje być znormalizowana przy wartościach:
# a= 1.01001100010011101001100*2^-126
# a= 0.10100110001001110100110*2^-126
# a= 0.01010011000100111010011*2^-126
# dzieje się tak, ponieważ cecha składa się już z samych 0 i liczba staje się zbyt mała, żeby ją zapisać jako liczbę znormalizowaną
# mantysa nie posiada już "domyślnego" pierwszego bitu jako 1, aby móc tą jedynkę przesunąć i uzyskać coraz mniejsze liczby
```

```c
#include <stdio.h>
#include <gsl/gsl_sf_bessel.h>
#include <gsl/gsl_ieee_utils.h>

int main ()
{
    float a = 1e-33;
    while (a>0) {
        a = a/2.0;
        printf (" a=");
        gsl_ieee_printf_float(&a);
        printf ("\n");
    }
    return 0;
}
```

```
Output:
 a= 1.01001100010011101001100*2^-111
 a= 1.01001100010011101001100*2^-112
 a= 1.01001100010011101001100*2^-113
 a= 1.01001100010011101001100*2^-114
 a= 1.01001100010011101001100*2^-115
 a= 1.01001100010011101001100*2^-116
 a= 1.01001100010011101001100*2^-117
 a= 1.01001100010011101001100*2^-118
 a= 1.01001100010011101001100*2^-119
 a= 1.01001100010011101001100*2^-120
 a= 1.01001100010011101001100*2^-121
 a= 1.01001100010011101001100*2^-122
 a= 1.01001100010011101001100*2^-123
 a= 1.01001100010011101001100*2^-124
 a= 1.01001100010011101001100*2^-125
 a= 1.01001100010011101001100*2^-126
 a= 0.10100110001001110100110*2^-126
 a= 0.01010011000100111010011*2^-126
 a= 0.00101001100010011101010*2^-126
 a= 0.00010100110001001110101*2^-126
 a= 0.00001010011000100111010*2^-126
 a= 0.00000101001100010011101*2^-126
 a= 0.00000010100110001001110*2^-126
 a= 0.00000001010011000100111*2^-126
 a= 0.00000000101001100010100*2^-126
 a= 0.00000000010100110001010*2^-126
 a= 0.00000000001010011000101*2^-126
 a= 0.00000000000101001100010*2^-126
 a= 0.00000000000010100110001*2^-126
 a= 0.00000000000001010011000*2^-126
 a= 0.00000000000000101001100*2^-126
 a= 0.00000000000000010100110*2^-126
 a= 0.00000000000000001010011*2^-126
 a= 0.00000000000000000101010*2^-126
 a= 0.00000000000000000010101*2^-126
 a= 0.00000000000000000001010*2^-126
 a= 0.00000000000000000000101*2^-126
 a= 0.00000000000000000000010*2^-126
 a= 0.00000000000000000000001*2^-126
 a= 0
```

Zadanie 4

Zadanie 4

Na przykładzie wybranego algorytmu niestabilnego numerycznie:

1. Pokazać, że działa źle.
2. Pokazać które konkretnie działania powodują zwiększenie błędu (np. dzielenie przez małą liczbę, cancellation).
3. Zademonstować wersję stabilną.

Wszystkie punkty przedstawić w postaci notatnika Julii.

]:
```
# Algorytm niestabilny numerycznie

# Zjawisko catastrophic cancellation (z wykładu):
# odejmowanie bliskich liczb - wynikiem jest mała liczba, mająca
# dużo zer tam, gdzie jej "poprzednicy" mieli cyfry znaczące,
# taka liczba jest normalizowana, zera zapisywane są w
# wykładniku (cecha), a cała zawartość mantysy przesuwana jest "w lewo"
# nie wiadomo czym zapełnić pojawiające się miejsca w
# mantysie po prawej stronie (zera lub przypadkowe wartości)

# Przykładem algorytmu niestabilnego numerycznie może być obliczanie wartości funkcji x^2-y^2.
# Gdy x i y mają bardzo małą różnicę, to możemy spotkać się z błędami w wynikach.

fun1(x::Float64, y::Float64) = x^2 - y^2

test_x = Float64(1+2^-30)
test_y = Float64(1+2^-31)

println("Wynik dla x^2 - y^2: ", fun1(test_x,test_y))
# tu bardzo dobrze widać błąd w wyniku (licząc za pomocą naszej funkcji inne rozwinięcie, bo napotkaliśmy zjawisko catastrophic cancellation)

# Wersja stabilna - wykonujemy operacje arytmetyczne na mniejszych liczbach (dzięki temu minimalizujemy błędy zaokrągleń)
fun2(x::Float64, y::Float64) = (x+y)*(x-y)

println("Wynik dla (x+y)(x-y): ", fun2(test_x,test_y))
```

```
# Dokładna wartość i porównanie wyników:
x = BigFloat(1+2^(-30))
y = BigFloat(1+2^(-31))
exact_result = x^2 - y^2
println("Wynik dokładny (obliczony na BigFloat): ", exact_result)

# Różnica w wynikach
println()
println("Różnica realnej wartości i wartości z fun1: ", exact_result - fun1(test_x,test_y))
println("Różnica realnej wartości i wartości z fun2: ", exact_result - fun2(test_x,test_y))
```

```
Wynik dla x^2 - y^2: 9.313225746154785e-10
Wynik dla (x+y)(x-y): 9.313225752659998e-10
Wynik dokładny (obliczony na BigFloat): 9.313225752659998191163026604044716805219650268554687 5e-10

Różnica realnej wartości i wartości z fun1: 6.505213034913026604044716805219650268554687 5e-19
Różnica realnej wartości i wartości z fun2: 0.0
```